Metro App – Flutter Version

Cairo Metro route planner built with Flutter.

GitHub repository: https://github.com/YOUR_USERNAME/metro_app

Features

- Enter start and end station: estimated time, station count, ticket price, direction and route.
- Multiple routes (up to 3): fastest, fewest transfers, fewest stations.
- Shortest route highlighted first (Dijkstra over station+line nodes, transfers handled correctly).
- Open the selected station in a map app.
- Nearest station to the user's current GPS location.
- Nearest station to an address the user types.
- Clean single-file code, small data set, no backend.

How it works

- Stations and coordinates are stored as compact strings per line and parsed once at startup.
- Graph nodes are (station, line). Riding to the next stop costs 2 min; changing line costs 5 min.
- The search runs with 3 transfer penalties to produce distinct alternatives, then sorts them by time.
- Ticket price is computed from the number of stations travelled (edit the `ticket` getter if prices change).
- Currently covers Line 1 and Line 2. Line 3 can be added as one extra entry in `_raw`.

Run the project

- `git clone https://github.com/YOUR_USERNAME/metro_app`
- `cd metro_app && flutter pub get`
- `flutter run`
- Android: add ACCESS_FINE_LOCATION, ACCESS_COARSE_LOCATION and INTERNET permissions to AndroidManifest.xml.
- iOS: add NSLocationWhenInUseUsageDescription to Info.plist.

Source: pubspec.yaml

```
name: metro_app
description: Cairo Metro route planner
publish_to: none
version: 1.0.0+1

environment:
  sdk: ">=3.0.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  geolocator: ^12.0.0
  geocoding: ^3.0.0
  url_launcher: ^6.3.0

flutter:
  uses-material-design: true
```

Source: lib/main.dart

```
import 'dart:math';
import 'package:flutter/material.dart';
import 'package:geocoding/geocoding.dart';
import 'package:geolocator/geolocator.dart';
import 'package:url_launcher/url_launcher.dart';

void main() => runApp(const MetroApp());

// ─────────────────────────── DATA ───────────────────────────
// Format: name|lat|lng  (coordinates are approximate)
// To add Line 3, just add another entry with the same format.
const _raw = <String, String>{
  'Line 1': 'Helwan|29.849|31.334;Ain Helwan|29.862|31.325;Helwan University|29.869|31.320;Wadi
    Hof|29.879|31.314;Hadayek Helwan|29.897|31.305;El-Maasara|29.906|31.299;Tora El-
    Asmant|29.926|31.287;Kozzika|29.936|31.281;Tora El-Balad|29.947|31.273;Sakanat El-
    Maadi|29.960|31.264;Maadi|29.960|31.257;Hadayek El-Maadi|29.970|31.251;Dar El-Salam|29.982|31.243;El-
    Zahraa|29.995|31.231;Mar Girgis|30.006|31.230;El-Malek El-Saleh|30.017|31.231;Al-Sayeda
    Zeinab|30.029|31.235;Saad
    Zaghloul|30.037|31.238;Sadat|30.044|31.234;Nasser|30.053|31.239;Orabi|30.057|31.242;Al-
    Shohadaa|30.061|31.246;Ghamra|30.069|31.264;El-Demerdash|30.077|31.277;Manshiet El-
    Sadr|30.082|31.281;Kobri El-Qobba|30.087|31.284;Hammamat El-Qobba|30.089|31.290;Saray El-
    Qobba|30.098|31.294;Hadayeq El-Zaitoun|30.106|31.298;Helmeyet El-Zaitoun|30.114|31.304;El-
    Matareyya|30.121|31.313;Ain Shams|30.131|31.319;Ezbet El-Nakhl|30.139|31.324;El-
    Marg|30.152|31.336;New El-Marg|30.163|31.339',
  'Line 2': 'Shubra El-Kheima|30.122|31.244;Kolleyyet El-
    Zeraa|30.114|31.249;Mezallat|30.105|31.246;Khalafawy|30.098|31.246;St. Teresa|30.088|31.246;Rod El-
    Farag|30.080|31.245;Massara|30.070|31.245;Al-Shohadaa|30.061|31.246;Attaba|30.052|31.247;Mohamed
    Naguib|30.045|31.244;Sadat|30.044|31.234;Opera|30.042|31.225;Dokki|30.038|31.212;El
    Bohoth|30.036|31.200;Cairo University|30.026|31.201;Faisal|30.017|31.204;Giza|30.007|31.207;Omm El-
    Masryeen|30.001|31.208;Sakiat Mekki|29.995|31.208;El-Mounib|29.981|31.212',
};

final Map<String, List<String>> lines = {};
final Map<String, List<double>> pos = {};

void _init() {
  _raw.forEach((line, s) {
    lines[line] = s.split(';').map((e) {
      final p = e.split('|');
      pos[p[0]] = [double.parse(p[1]), double.parse(p[2])];
      return p[0];
    }).toList();
  });
}

// ─────────────────────────── LOGIC ──────────────────────────
const _rideMin = 2, _transferMin = 5;

class Leg {
  final String line, from, to, dir;
  final int stops;
  Leg(this.line, this.from, this.to, this.dir, this.stops);
}

class Trip {
  final List<Leg> legs;
  Trip(this.legs);
  int get stops => legs.fold(0, (s, l) => s + l.stops);
  int get transfers => legs.length - 1;
  int get minutes => stops * _rideMin + transfers * _transferMin;
  // Verify current prices and edit here if they change.
  int get ticket => stops <= 9 ? 8 : stops <= 16 ? 10 : stops <= 23 ? 15 : 20;
```

```
  String get sig => legs.map((l) => '${l.line}${l.from}${l.to}').join('>');
}

/// Dijkstra over (station, line) nodes. [pen] = transfer penalty.
Trip? _route(String a, String b, int pen) {
  final dist = <String, int>{}, prev = <String, String>{}, done = <String>{};
  for (final l in lines.keys) {
    if (lines[l]!.contains(a)) dist['$a#$l'] = 0;
  }
  while (true) {
    String? cur;
    for (final e in dist.entries) {
      if (!done.contains(e.key) && (cur == null || e.value < dist[cur]!)) cur = e.key;
    }
    if (cur == null) return null;
    final c = cur;
    done.add(c);
    final st = c.split('#')[0], l = c.split('#')[1];
    if (st == b) {
      final path = <String>[];
      for (String? n = c; n != null; n = prev[n]) {
        path.add(n);
      }
      return _toTrip(path.reversed.toList());
    }
    void relax(String n, int cost) {
      final d = dist[c]! + cost;
      if (d < (dist[n] ?? 1 << 30)) {
        dist[n] = d;
        prev[n] = c;
      }
    }

    final i = lines[l]!.indexOf(st);
    if (i > 0) relax('${lines[l]![i - 1]}#$l', _rideMin);
    if (i < lines[l]!.length - 1) relax('${lines[l]![i + 1]}#$l', _rideMin);
    for (final o in lines.keys) {
      if (o != l && lines[o]!.contains(st)) relax('$st#$o', pen);
    }
  }
}

Trip _toTrip(List<String> path) {
  final legs = <Leg>[];
  int s = 0;
  String ln(int i) => path[i].split('#')[1];
  String st(int i) => path[i].split('#')[0];
  for (int i = 1; i <= path.length; i++) {
    if (i == path.length || ln(i) != ln(s)) {
      final list = lines[ln(s)]!;
      final dir = list.indexOf(st(i - 1)) > list.indexOf(st(s)) ? list.last : list.first;
      if (i - 1 > s) legs.add(Leg(ln(s), st(s), st(i - 1), dir, i - 1 - s));
      s = i;
    }
  }
  return Trip(legs);
}

/// Returns up to 3 distinct routes; first = fastest.
List<Trip> plan(String a, String b) {
  if (a == b) return [];
  final out = <String, Trip>{};
  for (final pen in [_transferMin, 30, 0]) {
    final t = _route(a, b, pen);
    if (t != null) out[t.sig] = t;
  }
  return out.values.toList()..sort((x, y) => x.minutes.compareTo(y.minutes));
}

String nearest(double lat, double lng) {
  double d(List<double> p) {
    const r = 0.017453292519943295;
    final x = (p[1] - lng) * r * cos((p[0] + lat) * r / 2), y = (p[0] - lat) * r;
    return x * x + y * y;
  }

  return pos.keys.reduce((a, b) => d(pos[a]!) < d(pos[b]!) ? a : b);
}

Future<void> openInMap(String station) {
  final p = pos[station]!;
  return launchUrl(
    Uri.parse('https://www.google.com/maps/search/?api=1&query=${p[0]},${p[1]}'),
    mode: LaunchMode.externalApplication,
  );
}
```

```
// ──────────────────────────── UI ────────────────────────────
class MetroApp extends StatelessWidget {
  const MetroApp({super.key});
  @override
  Widget build(BuildContext context) {
    _init();
    return MaterialApp(
      title: 'Cairo Metro',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.red, useMaterial3: true),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatefulWidget {
  const HomePage({super.key});
  @override
  State<HomePage> createState() => _HomePageState();
}

class _HomePageState extends State<HomePage> {
  final stations = (pos.keys.toList()..sort());
  String? from, to;
  List<Trip> trips = [];

  void _msg(String m) => ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(m)));

  Future<void> _nearMe() async {
    try {
      var perm = await Geolocator.checkPermission();
      if (perm == LocationPermission.denied) perm = await Geolocator.requestPermission();
      if (perm == LocationPermission.denied || perm == LocationPermission.deniedForever) {
        return _msg('Location permission denied');
      }
      final p = await Geolocator.getCurrentPosition();
      setState(() => from = nearest(p.latitude, p.longitude));
      _msg('Nearest station: $from');
    } catch (e) {
      _msg('Could not get location');
    }
  }

  Future<void> _nearAddress() async {
    final c = TextEditingController();
    final addr = await showDialog<String>(
      context: context,
      builder: (_) => AlertDialog(
        title: const Text('Enter address'),
        content: TextField(controller: c, autofocus: true),
        actions: [TextButton(onPressed: () => Navigator.pop(context, c.text), child: const
          Text('Find'))],
      ),
    );
    if (addr == null || addr.trim().isEmpty) return;
    try {
      final r = await locationFromAddress('$addr, Egypt');
      final s = nearest(r.first.latitude, r.first.longitude);
      if (!mounted) return;
      showDialog(
        context: context,
        builder: (_) => AlertDialog(
          title: Text('Nearest: $s'),
          actions: [
            TextButton(onPressed: () => openInMap(s), child: const Text('Open map')),
            TextButton(
              onPressed: () {
                setState(() => from = s);
                Navigator.pop(context);
              },
              child: const Text('Set as start'),
            ),
          ],
        ),
      );
    } catch (e) {
      _msg('Address not found');
    }
  }

  Widget _picker(String label, String? val, ValueChanged<String?> cb) => Expanded(
        child: DropdownButtonFormField<String>(
          value: val,
          isExpanded: true,
          decoration: InputDecoration(labelText: label, border: const OutlineInputBorder()),
          items: stations.map((s) => DropdownMenuItem(value: s, child: Text(s))).toList(),
```

```
          onChanged: cb,
        ),
      );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Cairo Metro')),
      body: ListView(padding: const EdgeInsets.all(12), children: [
        Row(children: [
          _picker('From', from, (v) => setState(() => from = v)),
          IconButton(
            icon: const Icon(Icons.swap_vert),
            onPressed: () => setState(() {
              final t = from;
              from = to;
              to = t;
            }),
          ),
          _picker('To', to, (v) => setState(() => to = v)),
        ]),
        const SizedBox(height: 8),
        Wrap(spacing: 8, children: [
          FilledButton.icon(
            icon: const Icon(Icons.alt_route),
            label: const Text('Find route'),
            onPressed: (from == null || to == null) ? null : () => setState(() => trips = plan(from!,
              to!)),
          ),
          OutlinedButton.icon(icon: const Icon(Icons.my_location), label: const Text('Nearest to me'),
            onPressed: _nearMe),
          OutlinedButton.icon(icon: const Icon(Icons.search), label: const Text('Nearest to address'),
            onPressed: _nearAddress),
        ]),
        const SizedBox(height: 8),
        Row(children: [
          if (from != null) TextButton(onPressed: () => openInMap(from!), child: const Text('Map:
            start')),
          if (to != null) TextButton(onPressed: () => openInMap(to!), child: const Text('Map: end')),
        ]),
        for (int i = 0; i < trips.length; i++) _tripCard(trips[i], i),
      ]),
    );
  }

  Widget _tripCard(Trip t, int i) => Card(
        color: i == 0 ? Colors.green.shade50 : null,
        child: Padding(
          padding: const EdgeInsets.all(12),
          child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
            Text(i == 0 ? 'Shortest route' : 'Alternative $i', style: const TextStyle(fontWeight:
              FontWeight.bold)),
            Text('Time: ${t.minutes} min  |  Stations: ${t.stops}  |  Transfers: ${t.transfers}  |
              Ticket: ${t.ticket} EGP'),
            const Divider(),
            for (final l in t.legs)
              ListTile(
                dense: true,
                contentPadding: EdgeInsets.zero,
                leading: const Icon(Icons.subway),
                title: Text('${l.line}: ${l.from} → ${l.to}'),
                subtitle: Text('Direction: ${l.dir} • ${l.stops} stops'),
              ),
          ]),
        ),
      );
}
```